

Universidad San Carlos de Guatemala
Facultad de Ingeniería.
Ingeniería en ciencias y sistemas



FIUSAC
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



PROYECTO 2

PytoGO

La presente plantilla es una guía para la redacción de un proyecto y las secciones pueden ser modificadas o eliminadas según el tipo de proyecto.

PONDERACIÓN: 40 pts

 **Tiempo estimado: 40 hrs**

Índice

1. MARCO FORMATIVO.....	3
1.1. Valor.....	3
1.2. Competencia(s).....	3
1.3. Objetivo SMART.....	3
2. Resumen Ejecutivo.....	4
3. Enunciado del Proyecto.....	4
3.1 Descripción del problema a resolver.....	4
3.2 Alcance del proyecto.....	4
3.3 Entregables.....	5
4. Material de apoyo.....	7
5. Metodología.....	7
6. Recursos y herramientas a utilizar.....	8
7. Desarrollo de Habilidades Blandas.....	8
8. Cronograma.....	9
9. Rúbrica de Calificación.....	10
9.1 Requisitos para optar a la calificación.....	10
9.2 Resumen de Puntuaciones.....	12
9.5 Comentarios Generales.....	15
Detalle de la Calificación.....	15

1. MARCO FORMATIVO

1.1. Valor

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Compromiso	El compromiso es esencial en el estudio de lenguajes formales porque este campo requiere dedicación constante para dominar conceptos abstractos y secuenciales.

1.2. Competencia(s)

Indique la competencia general del curso y la competencia específica trabajada en esta lectura.

Tipo de Competencia	
Competencia General	Analizar la evolución y clasificación de los lenguajes formales y de programación mediante la comparación de sus características, estructuras y herramientas para comprender su organización y funcionamiento
Competencia Específica	Generar gramáticas libres de contexto mediante el análisis sintáctico para reconocer lenguajes formales.

1.3. Objetivo SMART

Define el/los objetivo(s) que se pretende cumplir con la tarea planteada, con un enfoque SMART, toma en consideración que un objetivo SMART, va enfocado a la habilidad(y no a tanto a la teoría

SMART	Definición	Objetivo redactado
Específico (¿Qué?)	El objetivo es concreto y tangible.	Los estudiantes serán capaces de realizar gramáticas libres de contexto y realizar el análisis sintáctico
Medible (¿Cuánto?)	El objetivo tiene una medida objetiva de éxito.	Lograr analizar el lenguaje mínimo al 80% de las instrucciones dadas
Alcanzable (¿Cómo?)	El objetivo debe ser posible con los recursos disponibles.	Utilizando lenguajes de programación orientados a objetos (Java, Python o C#), bibliotecas para interfaces gráficas (Swing, JavaFX, Tkinter, PyQt), expresiones regulares nativas del lenguaje, y las herramientas de desarrollo disponibles en el laboratorio o mediante instalación gratuita.

Realista (¿Para qué?)	El objetivo contribuye a metas más amplias.	con habilidades prácticas que fortalezcan la formación en informática y sistemas, preparándose para cursos más avanzados en redes y seguridad.
A Tiempo (¿Cuándo?)	El objetivo tiene fecha límite o mejor aún un cronograma de hitos de progreso	Cumplir este aprendizaje en un plazo máximo de 4 semanas.

2. Resumen Ejecutivo

3. Enunciado del Proyecto

3.1 Descripción del problema a resolver

Traducir código manualmente entre lenguajes de programación es una tarea tediosa, propensa a errores y consume tiempo valioso de los desarrolladores. La falta de herramientas automatizadas confiables para convertir código Python a Go limita la reutilización de componentes existentes y dificulta la migración de proyectos entre estos lenguajes. Este proyecto busca desarrollar un traductor automático que analice código Python válido y lo transforme en su equivalente en Go, manteniendo la funcionalidad y reduciendo el trabajo manual de conversión.

3.2 Alcance del proyecto

Alcance obligatorio:

- Reconocer y traducir estructuras de control básicas (if, for, while) de Python a Go
- Convertir definiciones de funciones y llamadas de función
- Traducir tipos de datos primitivos (int, str, float, bool) con sus conversiones correspondientes
- Generar código Go compilable y ejecutable
- Documentar el mapeo de sintaxis Python → Go utilizado

Alcance opcional:

- Soporte para clases y programación orientada a objetos
- Manejo de excepciones (try/except en Python → error handling en Go)

- Traducción de comprensiones de listas y diccionarios
- Optimizaciones automáticas del código generado

Lenguaje de desarrollo: Python (para implementar el traductor)

Herramientas de análisis: Flex/Bison o librerías equivalentes (ply, antlr4) para parsing

Lenguaje destino: Go 1.16+

Control de versiones: GitHub

Documentación: Manual técnico en LaTeX o Markdown

3.3 Entregables

Tipo	Descripción
Traductor Ejecutable	Script Python que reciba código Python como entrada y genere código Go compilable.
Código Fuente	Código completo con estructura modular (lexer, parser, generador de código), comentado y con buenas prácticas.
Documentación Técnica	Manual técnico con descripción de la arquitectura, funciones implementadas y gramática utilizada.
Casos de Prueba	Conjunto de programas Python de prueba con sus equivalentes en Go validados.
Plan de Pruebas	Informe detallado con casos de uso, resultados y limitaciones identificadas.
Manual de Usuario	Guía de instalación, uso del traductor y ejemplos de ejecución.
Presentación Final	Presentación con demostración viva del traductor y resultados obtenidos.
Diagrama de Flujo	Representación gráfica del proceso de traducción (análisis léxico → sintáctico → generación).

4. Metodología

Fase 1: Investigación preliminar (Semana 1)

- Analizar la sintaxis y semántica de Python y Go
- Investigar herramientas de parsing (Flex/Bison, PLY, ANTLR)
- Estudiar diferencias clave entre ambos lenguajes
- Definir reglas de traducción básicas

Fase 2: Diseño del sistema (Semana 2)

- Crear diagramas de flujo del traductor (entrada → análisis → generación → salida)
- Diseñar la gramática formal para Python
- Definir tablas de mapeo sintáctico Python → Go
- Documentar la arquitectura en tres módulos: lexer, parser, code generator

Fase 3: Desarrollo (Semana 3-4)

- **Módulo Lexer:** Tokenización de código Python
- **Módulo Parser:** Análisis sintáctico y construcción del AST
- **Módulo Generador:** Recorrido del AST y generación de código Go
- Implementar por fases: variables → funciones → control de flujo

Fase 4: Pruebas y ajustes (Semana 5)

- Validar que el código Go generado compile correctamente
- Ejecutar casos de prueba y comparar resultados
- Documentar fallos y limitaciones
- Optimizar generación de código

Fase 5: Documentación y presentación (Semana 6)

- Completar documentación técnica y manual de usuario
- Grabar video demostrativo del traductor
- Preparar presentación final con ejemplos reales

6. Recursos y herramientas a utilizar

Software:

- Python 3.8+
- Go 1.16+
- PLY (Python Lex-Yacc) o antlr4-python3-runtime para parsing
- Git/GitHub para control de versiones

Plataformas:

- GitHub (repositorio con código fuente)
- Documentación en Markdown o LaTeX

Lecturas recomendadas:

- Documentación oficial: <https://docs.python.org> y <https://golang.org/doc>
- "Crafting Interpreters" - Robert Nystrom
- Tutoriales de PLY: <https://www.dabeaz.com/ply/>
- Comparativa Python vs Go: <https://yourbasic.org/golang/>

7. Desarrollo de Habilidades Blandas

- **Autogestión del tiempo:** Cumplir con los hitos de cada fase y entregas parciales
- **Responsabilidad:** Documentar decisiones de diseño y justificar limitaciones del traductor
- **Pensamiento crítico:** Identificar casos complejos y proponer soluciones creativas para traducción de características avanzadas
- **Comunicación técnica:** Redactar documentación clara que explique cómo funciona el traductor

8. Cronograma

Tipo	Fecha Inicio	Fecha Fin	Descripción
Asignación de Proyecto	Sem 1	Sem 1	Entrega del enunciado y recursos
Entrega 1	Sem 1	Sem 2	Investigación preliminar + Diseño del sistema
Entrega 2	Sem 2	Sem 4	Módulos Lexer, Parser y Generador (80% completado)
Entrega 3	Sem 4	Sem 5	Pruebas, validación y casos de uso documentados
Calificación	Sem 6	Sem 6	Evaluación final con presentación y defensa

9. Rúbrica de Calificación

9.1 Requisitos para optar a la calificación

Tema	Descripción	Cumple (Sí/No)
Cumplimiento de la tecnología establecida	El traductor debe estar implementado en Python utilizando herramientas de parsing (PLY, Flex/Bison o equivalente)	
Uso de herramientas requeridas	El proyecto debe compilar código Python válido y generar código Go ejecutable sin errores críticos	
Gestión y entregas de práctica	El repositorio en GitHub debe contener el código fuente, README, documentación y entregas parciales según cronograma	
Documentación obligatoria	Se debe incluir Manual Técnico, Manual de Usuario, Diagrama de Flujo y Plan de Pruebas en PDF	
Pruebas y funcionalidad mínima	El traductor debe traducir correctamente: variables, funciones, ciclos (for/while) e condicionales (if/else)	

9.2 Resumen de Puntuaciones

Área	Puntos Totales	Puntos Obtenidos
1. Habilidades		
Documentación Técnica (arquitectura, gramática, decisiones)	10	

Código Fuente (modularidad, legibilidad, comentarios)	12	
Entregas & Gestión de Repositorio (entregas a tiempo, estructura)	10	
Presentación Final (claridad, demostración, respuestas)	8	
Sub-Total Habilidades	40	
2. Conocimiento		
Análisis Léxico (tokenización correcta de Python)	12	
Análisis Sintáctico (AST válido, manejo de errores)	15	
Generación de Código Go (traducciones funcionales)	20	
Funcionalidad: Variables, Funciones y Control de Flujo	13	
Sub-Total Conocimiento	60	
TOTAL	100	

9.5 Comentarios Generales

Detalle de la Calificación

La presente tabla sirve como instrumento para evaluar el proyecto según criterios, niveles de desempeño y puntajes obtenidos. Cada fila corresponde a un aspecto a evaluar y debe completarse de la siguiente forma:

1. No. / Criterio de evaluación
 - a. Identifica el criterio y subcriterio específico que se está evaluando (ej. 1.1 Cumplimiento de requisitos).
2. Nivel de evaluación (Satisfactorio, Necesita mejorar)
 - a. En cada subcriterio, se debe seleccionar el nivel que mejor describa el desempeño del estudiante o del proyecto:
 - i. Satisfactorio (100% – 61%): Cumple plenamente con lo esperado.
 - ii. Necesita mejorar (60% – 0%): No alcanza los requisitos o presenta fallas significativas.
3. Descripción de cada nivel
 - a. Se incluyen en la tabla ejemplos de lo que significa alcanzar cada nivel para orientar la evaluación.
4. Nota obtenida
 - a. Según el nivel asignado, se debe registrar la calificación correspondiente en esta columna.
 - b. Puede usarse una escala numérica previamente definida (ej. de 0 a 100), tomando en cuenta los rangos de porcentaje establecidos.
5. Sub-Total de puntos
 - a. Al finalizar cada sección (Conocimientos, Procedimiento y Desarrollo, Innovación y Habilidades), se suman las notas obtenidas en los subcriterios.
6. Resultado final
 - a. La suma de todos los subtotales corresponde a la nota global de la evaluación.

Para conocer más detalles sobre la elaboración de rúbricas, puedes consultar el siguiente video::

https://drive.google.com/file/d/1Csljq6yWUmByzyl2D3oizGgbDXXX_WC3/view?usp=sharing

Ejemplo para curso de área Software – Introducción a la Programación 1

	Criterio/Nivel			Nivel de evaluación		
No.	Criterio de evaluación	Punteo maximo		Satisfactorio 100% - 61%	Necesita mejorar 60% - 0%	Punteo Obtenido
1	Habilidades	40				
1.1	Uso de ciclos y estructuras (Vectores)	8	Descripción	El programa utiliza de forma correcta ciclos y estructuras de datos (vectores) para resolver los problemas planteados, optimizando la lógica y garantizando resultados coherentes.	El uso de ciclos y estructuras es inadecuado, genera errores de ejecución o no resuelve los problemas correctamente, evidenciando un manejo insuficiente de vectores.	
			Rango del punteo	8 - 5/	4 - 0/	
	Sub-Total de Puntos					
2	Conocimientos	60				

2.1	Gestión de Personajes (CRUD con validaciones)	30	Descripción	El sistema permite crear, leer, actualizar y eliminar personajes de manera completa, aplicando validaciones que impiden datos duplicados o inconsistentes. Todas las operaciones se ejecutan sin errores y mantienen la integridad de la información.	El sistema presenta fallos en las operaciones de CRUD, carece de validaciones básicas o permite el ingreso de datos inválidos, generando inconsistencias en la información de los personajes.	
			Rango del punteo	30 - 18/	17 - 0/	
	Sub-Total de Puntos					
	Total					

Ejemplo de rúbrica completa: [➤ Rubrica Computacion](#)

Ejemplo para curso de área Sistemas – Sistemas Organizacionales y Gerenciales 1

	Criterio/Nivel			Nivel de evaluación		
No.	Criterio de evaluación	Punteo maximo		Satisfactorio 100% - 61%	Necesita mejorar 60% - 0%	Punteo Obtenido
1	Habilidades	40				

1.1	Informe Ejecutivo (redacción, análisis y justificación)	10	Descripción	El informe presenta una redacción clara, análisis adecuado y justificación lógica.	El informe es superficial, con redacción deficiente, sin análisis o sin justificación válida.	
			Rango del punteo	10 - 6/	5 - 0/	
	Sub-Total de Puntos					
2	Conocimientos	60				
2.1	Script de Base de Datos (modelo correcto, ejecución sin errores)	25	Descripción	El script cumple con la creación de todas las tablas, relaciones y restricciones sin errores, ejecutándose de forma correcta.	El script presenta errores de sintaxis o diseño, impide la ejecución o no cumple con la creación de todas las tablas y relaciones.	
			Rango del punteo	25 - 15/	14 - 0/	
	Total					

Ejemplo de plantilla completa: [📄 Rubrica Sistemas](#)

Ejemplo para curso de área Computación – Organización Computacional

	Criterio/Nivel			Nivel de evaluación		
No.	Criterio de evaluación	Punteo maximo		Satisfactorio 100% - 61%	Necesita mejorar 60% - 0%	Punteo Obtenido
1	Habilidades	40				
1.1	Documentación (PDF: K-map, funciones, diagramas, etc.)	10	Descripción	La documentación está completa, organizada y presenta los elementos solicitados con claridad.	La documentación es incompleta, desordenada o carece de los elementos requeridos (K-map, funciones, diagramas).	
			Rango del punteo	10 - 6/	5 - 0/	
	Sub-Total de Puntos					
2	Conocimientos	60				
2.1	Funcionamiento correcto – Display Cátodo Común	25	Descripción	El display funciona de manera correcta, mostrando todos los segmentos y el punto decimal sin errores.	El display no enciende adecuadamente, muestra segmentos erróneos o no cumple con la lógica de funcionamiento.	
			Rango del punteo	25 - 15/	14 - 0/	

	Sub-Total de Puntos					
	Total					

Ejemplo de plantilla completa: [+ Rubrica Computacion](#)